

ELI MKXI — Current State Snapshot (2026-06-07)

Contents

Scale (measured 2026-06-08)	1
Tests & eval (measured on .venv, 2026-06-08)	1
GPU / inference (corrected this session)	2
What this session added / fixed (all on main, green)	2
Honest weaknesses (unchanged; from complete_findings.md)	2
Addendum (2026-06-07, later)	2
Addendum (2026-06-07, DAG + delegation)	3
Addendum (2026-06-08, routing/scheduling/multi-command)	3
Addendum (2026-06-08, later — reliability pass + voice/wake/tone + cognition correctness)	3
Addendum (2026-06-08, latest — media control, self-heal escalation, Full Control)	4
Update Advisory — 2026-06-09 (live-session bug-fix pass: routing, habits, terminal)	5
Update Advisory — 2026-06-08 (autonomy, persona voice, full-brain audit)	6
Verdict	6
Update — 2026-06-09 (examiner/grounding correctness; VRAM; clean shutdown)	6

Authoritative, freshly-measured summary of the whole project after this session's work. Numbers are measured on ELI's real interpreter (.venv/bin/python), not the agent's system python. Deeper detail lives in the companion blueprints (project_overview.md, architecture.md, orchestration_and_agents.md, grounding_and_evidence.md, capabilities_and_actions.md, complete_findings.md).

Scale (measured 2026-06-08)

- **133,430 LOC** across **352** .py files under eli/.
- **206** manifest capabilities (capability_manifest.json): **166** in the executor SUPPORTED_ACTIONS, plugin-backed remainder.
- **14 main GUI tabs** (Chat, Proactive, Images, Quick Actions, Screen, Files, Labs, Coding, Tasks, Report Builder, Test & Review, Orchestration, Eli's World, Settings); **14** bus agents + the CodeAgent; **4** SQLite stores (user / agent / system_index / coding_memory).
- **151** test files.
- (Earlier baseline: 129,955 LOC / 346 files / 196 caps / 12 tabs.)

Tests & eval (measured on .venv, 2026-06-08)

- **6,630 tests collected; 6,586 passed, 0 failed, 2 xfailed, 42 skipped** (~6m47s on the GPU .venv). Was 6,415 collected / 6,371 passed.
- Composition: original unit/regression/integration (~2,350) + **tests/claims/** (~3,970 — examines the project vs its claims: every module compiles + core imports; every manifest capability well-formed + flags match the live executor; every SUPPORTED_ACTION handled; every documented action reachable; activation phrases; blueprint refs resolve; structural/behavioural claims; **symbol inventory** = every public function/class/method is a real introspectable callable) + the meta-tests for the doc pipeline, evidence planner, and test generator + generated tests.
- **Eval harness:** 41 cases = **38 router (auto-run under pytest)** + **10 engine (model-backed)**. The engine board + full report now refresh **nightly** (scheduled eval task), as does ELI-assisted test generation (testgen).

- **2 xfail = documented, intentional:** MOUSE_CONTROL bare “left click” → GAZE_CLICK by design; one grounding-mode case.

GPU / inference (corrected this session)

ELI runs on the **GPU**. `.venv/bin/python → llama_cpp.llama_supports_gpu_offload() = True` (llama-cpp-python 0.3.23, CUDA; driver 595.71.05, CUDA 13.2, nvcc 12.8). Model loads with `ggml_cuda_init: found 1 CUDA device`, layers offloaded to the 2060 Super; a short generation ≈ 1.2 s. (An earlier “CPU-only” finding was a sandbox error — the agent’s bare python is the *system* interpreter with a CPU wheel, not the venv ELI uses. No rebuild needed.)

What this session added / fixed (all on main, green)

- **Grounded generation:** evidence-planner (plan→gather→consume across code/web/ memory/runtime agents) + multi-stage report_pipeline (outline→sections→review) for documents/scripts/projects, with confidence-driven deeper-tier re-gather.
- **Grounded perception/introspection:** “what’s on screen” → real vision glance; identity/awareness/cognition queries gather-then-summarise (no data dumps, no weights-only answers).
- **Autonomy/self-awareness tick** wired into the proactive daemon (governed, 30-min: code-monitor + self-model refresh + goal/scheduler proposals).
- **Self-correctness loop:** auto test report on every pytest run (artifacts/test_report.md); RUN_TESTS + GENERATE_TESTS actions ELI can run and summarise; the report is an evidence channel for upgrade proposals; nightly eval + testgen scheduled (durable + recurring).
- **GUI:** Report Builder promoted to a main tab; Files-tab document converter (pdf/docx/doc/odt/rtf/html/md/tex/e + lualatex).
- **Fixes:** habit 00:00 offer; EXAMINE_CODE↔GUI_RUNTIME_AUDIT route collision; grounding quick-mode hedge regression; 4 router-gap fixes; venv test-collection error; scheduled-store test isolation. All prior known failures resolved.

Honest weaknesses (unchanged; from complete_findings.md)

- **God-files:** `executor_enhanced.py`, `engine.py` (~12k each) + the GUI (~10k).
- **~2,565 swallowed except Exception** — failures are absorbed, not surfaced.
- **~15 load-bearing globals()/install-time wrappers** (netguard, adaptive GGUF loader, middleware table) — architecture, not test-maskers, but a fold-in target.
- **Duplication:** two image engines; overlapping `runtime/*_response/*_surface`.
- **Repo hygiene:** root junk (... , one-off `patch_*.py`, `verify_*.sh`, zips).
- **The real ceiling is the local model** — the body/memory/grounding/awareness are frontier; the *mind* is whatever local GGUF is loaded (model-agnostic → a swap).

Addendum (2026-06-07, later)

- **LoRA wired** (was orphaned): model-agnostic target modules + `lora_pipeline` DAG (pre-flight→build→train→eval) + LORA_STATUS/LORA_TRAIN actions + scheduled lora kind. See `lora_pipeline.md`.
- **Doc “regression” was test pollution, not quality:** a doc-gen test leaked its stub into the real artifacts/documents/. Root cause: `runtime_settings` strips out-of-project `ELI_ARTIFACTS_DIR`. Fixed: `_artifacts_dir()/_store_path()` honour it for in-project paths; `confstest` redirects all test artifact writes to an in-project throwaway dir (one config line, **no monkeypatch**).
- **Monkeypatches:** removed the test fixtures previously added; the only remaining runtime self-wrappers are 2 in `gguf_inference` (the generate auto-reload wrap and the effective-runtime `load_model` contract) — verified **load-bearing** (a plain rebind of generate recurses infinitely), so they are module-init wiring, not removable hacks.
- **Stale-folder audit (eli/):** `guards/` is dead (empty, 0 importers — removable); `brain/agents/` is the runtime custom-agent store; `scripts/` is CLI-only; `integrations/cli/contracts/system/utis/world`

all import cleanly and are wired (utils 59 importers, kernel 24) — “stale” = unchanged stable code.

- **Open (large, dedicated efforts):** a one-click professional installer (frozen venv + model/DB/repo provisioning) and elevating the DAG to a project-wide frontier orchestrator.

Addendum (2026-06-07, DAG + delegation)

- **DAG orchestrator** (eli/core/dag.py): pure scheduler → full execution engine (parallel layers, retries, fallback, cache, conditional, timeout, priority, fail-fast, budget, telemetry). Wired: the 14 agents run on it; evidence gather runs its channels in parallel on it; coding planner + LoRA pipeline already use it. See dag_orchestrator.md.
- **Capabilities: 200** (added RUN_TESTS, GENERATE_TESTS, LORA_STATUS/LORA_TRAIN, ORCHESTRATION_STATUS, TEST_REVIEW).
- **GUI: 12 main tabs** (Report Builder is top-level; **Test & Review** + **Orchestration** are **Labs sub-tabs** — demoted 2026-06-18 as dev tools; Labs = 10 sub-tabs). See gui.md.
- **Test & Review** (runtime/test_review.py + TEST_REVIEW): full suite run → LLM summary → prior report backed up + errors file written → result-driven options. On failure it **delegates analysis to the existing code examiner, orchestrated over the failing modules in parallel via run_graph** — the intended pattern: *compose existing functions/agents, don't rebuild*. Many other actions can adopt the same delegation (candidates: SELF_IMPROVE→coding agent, GENERATE_PROJECT→planner DAG).

Addendum (2026-06-08, routing/scheduling/multi-command)

- **Scale now:** ~131k LOC / **349** .py files; **201** manifest capabilities; ~**6,508** tests passing (6,552 collected).
- **Compound “schedule + action” fixed:** “get/set a morning report for 7:15 tomorrow” now SCHEDULES (was running now). Generalised: **any imperative command + a future time** → background workers (“open spotify at 8pm”, “get the news at 7am”, “close steam in 2 hours”). schedule_prepass runs before the app router; excludes alarms/timers (own handlers) and questions; the re-run request is the clean, time-stripped command (runs once, never re-schedules). Recurring on “every/daily”.
- **Multi-command chaining:** command_splitter.split_commands + router_multi_command_prepass → MULTI_COMMAND action + executor handler routes & executes each segment in order (“close steam and set an alarm for 7am”). Works for voice (GUI_DIRECT_EXEC) and typed (process). Strongly guarded against over-splitting. Details in runtime_planning_world.md + dag_orchestrator.md.

Addendum (2026-06-08, later — reliability pass + voice/wake/tone + cognition correctness)

Scale now: 133,430 LOC / **352** files / **205** capabilities (166 SUPPORTED_ACTIONS) / 151 test files. New actions: WAKE_TRAIN, WAKE_ENROLL, WAKE_SET, TRAIN_VOICE. New subsystems: eli/perception/wakeword.py, eli/perception/voice_profile.py; rewritten eli/cognition/llm_intent.py.

Crash/correctness fixes (all tested): - **MULTI_COMMAND UnboundLocalError** — a LORA_TRAIN local execute= shadowed the module dispatch in _execute_impl, so MULTI_COMMAND (and any in-function re-dispatch, e.g. “what time is it”) hit an unbound local. Renamed the local. (executor_enhanced.) - **FIX_FILE backup crash** — a bare from datetime import datetime in the TIME branch shadowed datetime function-wide; FIX_FILE's backup builder hit an unbound local. Aliased it. (executor_enhanced.) - **LoRA build_job x30 error** — the trainer only *validated* the dataset, never built it. build_training_job now calls dataset_builder.build_dataset when missing (615 rows here, was 0), and “no curated/reviewed data yet” is reclassified as a benign not-ready state (blocks will_train, no longer a logged error). (lora_trainer, lora_pipeline.) See lora_pipeline.md.

Routing by intelligence, not hardcodes — the regex router returns fallback.chat at conf 0.6, which cleared the engine's >0.5 gate, so the existing LLM-intent fallback was dead and unmatched phrasings dropped to a blind CHAT that couldn't act (and hallucinated facts like the date). engine._parse_intent now treats fallback.chat as *unmatched*; llm_intent was rewritten to resolve

against ELI's **real SUPPORTED_ACTIONS catalogue** (validated, args extracted, CHAT default). Live 13/13 on the near-miss class (date→DATE, "set the volume to 40%"→VOLUME, hub/dictation/gaze/workspace/code_solve/data_fabricator). No phrasing regexes added.

Mode-gated determinism (corrects the "LLM bypass" overstatement) — the deterministic direct-return is **partial and mode-gated**, NOT a blanket bypass: router-fast/command actions return the executor result **verbatim in quick mode** and **synthesise in non-quick modes** (engine._deterministic_direct_payload_actions + _bypass_persona). The command actions (WRITE_NOTE, VOLUME, window/media/file/time) were in the wrong set, so quick mode re-synthesised them — corrupting results ("Wrote note"→"Bought note") and adding latency. Fixed by set membership. (See the correction note in grounding_and_evidence.md.)

Vision VRAM cliff (A3) — after a vision hot-swap the text-model restore started from the raw oversized request (ctx=30720) and adaptively collapsed to gpu_layers=16 for the rest of the session. The adaptive loader now tries the **last known-good published config first** (_live_runtime_override, e.g. 22528/99), restoring full-GPU speed. Portable — reuses whatever the hardware profiler computed for *this* machine. See inference_and_hardware.md.

Voice / STT / wake word / tone (new): - Duration-adaptive STT pause — short commands finalise after 0.5s of silence (was ~1.4s); a prompt past 12s needs 2s of silence so long dictation isn't cut. Faithful copy of sr's capture with one dynamic condition; flag-gated fallback. (audio_stt.) - **Self-trained, fully-local wake word robust over music** (wakeword.py) — ELI synthesises the wake phrase with its OWN Piper TTS across voices/speeds, mixes it with noise/music at random SNRs (the augmentation = robustness over music), embeds via openWakeWord's open extractor, and trains a small classifier head it owns. No account, no third-party, no unavailable pre-trained model. Used in the unarmed loop to catch the wake word even when whisper transcribed the music; fallback-safe. - **User-settable wake word** — "change the wake word to " (WAKE_SET) persists any phrase, feeds both the acoustic model and the transcription matcher. - **Voice enrollment** — "enroll my wake word" folds the user's real voice into the wake model. - **Voice-profile subsystem** (voice_profile.py, SEPARATE from wake) — the foundation for tone/emotion. Real today (numpy): autocorrelation pitch track, energy, rate, and **question-vs-statement** from terminal-pitch slope; a **labelled-emotion** nearest- centroid classifier (neutral/happy/angry/excited/sad) trained from "train my voice" prompts. TRAIN_VOICE runs one unified guided session (wake reps + emotion-labelled lines). **Wired into cognition:** STT publishes the per-turn tone on a fresh side- channel; engine._build_enhanced_system injects a speaker-voice cue so ELI adapts its delivery to how the user sounds. See perception.md.

Media + installer: - "play X" honesty — when yt-dlp+mpv are absent (so it can only open a browser search), the result is marked played=False/search_only and says what to install, rather than claiming it played. (Spotify specific-track auto-play is a genuine no-Web- API limit.) - **Installer runtime tools** — install.sh now best-effort installs mpv/yt-dlp/ playerctl/wmctrl/xdotool/ffmpeg + xclip/wl-clipboard. See installation.md.

Addendum (2026-06-08, latest — media control, self-heal escalation, Full Control)

Scale now: 133,430 LOC / **352** files / **206** capabilities (167 SUPPORTED_ACTIONS) / 151 test files. New file: eli/core/full_control.py. New action: NOW_PLAYING. Per-package LOC/file table refreshed in architecture.md.

- **Spotify play regression fixed.** A 2026-06-06 change had switched the Spotify path from "play the requested SONG" to "play a playlist" (which never started playback). Reverted to the song-match approach (spotify:search:<q> → Play top song). "play X by Y on spotify" plays the song again.
- **Headless-YouTube (mpv) control + now-playing.** "Play on YouTube" runs a HEADLESS mpv (bestaudio, no window) that playerctl can't see, so pause/stop/resume couldn't reach it. Added mpv-IPC control (_mpv_ipc/_mpv_alive/_mpv_quit) + a media-state tracker (source/title); pause/stop/resume now route to mpv when YouTube is active, else playerctl; switching sources stops the old mpv. New **NOW_PLAYING** action ("what's playing") reports the track + source. See perception.md / capabilities_and_actions.md.

- **Self-heal escalation (recurring-error → proactive conversation).** Built on the existing `self_improvement.log_failure` 5× clause (which only logged to debug): ≥5× queues a user-facing notice; ≥10× also runs `analyze/patch` and records the `OUTCOME`. The engine (`_build_enhanced_system`) pops the top notice once per conversational turn and has ELI raise it with the user, then answer — recurring problems are surfaced IN conversation. Applying code patches stays gated behind `auto_patch_enabled` OR Full Control.
- **Two recurring health-probe errors root-caused** (not silenced): “No commands to run.” and “Missing description for `GENERATE_PROJECT`” were incomplete-INPUT cases wrongly returned as hard faults (polluting the failure table, raising `repair_pressure`). Now graceful clarifications (`ok=True`, `fault=False`, `needs_input=True`).
- **Ollama embed call killed.** `memory/habits_memory_db.embed()` POSTed to Ollama (:11434) for embeddings on every habit write — the recurring `HTTPConnectionPool(... 11434)` error, and a violation of offline-by-default. Now gated behind `provider=='ollama'`; otherwise the offline hashed embedding (which it already fell back to). No network call.
- **Stray ~/Desktop/artifacts/proactive** — `proposal_queue.py/proposal_adapters.py` used `parents[3]` (one level too high → the project's *parent*). Fixed to `paths.proactive_dir()`.
- **ELI Full Control** (`eli/core/full_control.py`) — a master override, **default OFF, single source of truth = the full_control setting / GUI toggle (no env var)**. When ON it lifts every barrier that consults `is_full_control()`: `netguard` (network), `approval_engine` (autonomy), self-patch gate, and the executor/security command-safety floor (destructive block + denylist). Red GUI toggle by the Net toggle with a confirmation dialog. See `security.md / gui.md`.
- **Reasoning-mode names:** the old internal labels (Chain of Thought / Tree of Thoughts / Constitutional AI / Self-Consistency) no longer leak to the user — every user-facing surface shows Quick/Normal/Advanced/Research/Expert. **JARVIS / AGI framing removed** from all code
 - blueprints.

Update Advisory — 2026-06-09 (live-session bug-fix pass: routing, habits, terminal)

Scale now ~**134.0k LOC / 353 files / 152 test files / 206 capabilities / 14 bus agents** (+`eli/planning/goal_autogenesis.py` and its test since the 2026-06-08 line). Fixes landed from a real chat session on a GPU-fitting 7B model (all on main, tested): - **Proactive-habits ↔ chat-agent disconnect (the headline).** The proactive daemon fills the habits table (`detect_habits`), but the bus **HabitAgent** read only the empty `habit_rules` table — so the chat path was blind to ELI's real behaviour. Now **HabitAgent** reads `get_detected_habits()`, `AgentResult.has_evidence` recognises it, and `DispatchResult.to_context_block` renders it — so a chat about routines is grounded in real patterns (“screenshot 41×, play media 38×...”). Completes the habit wiring across all three readers (`HABIT_STATUS`, overlay, chat agent). - **Routing/parsing cluster:** “open home folder” no longer tries to *install* an app and ~ now expands (`OPEN_FILE_SYSTEM`); “play X on youtube **and** close spotify” splits into `MULTI_COMMAND` (media post-contract no longer collapses an already-split chain); banter (“use your imagination!”) downgrades from a mis-guessed `GENERATE_SCRIPT` to `CHAT`; alarm parsing handles “7am tomorrow”/“07:00 hours”/“noon”/24h (was `int()` on the raw string). - **Symbolic world:** “which room are you in?” routes to `CHAT` (so the handoff's LIVE current room is used) instead of mis-routing to `GAZE_STATUS` where the model defaulted to “Core Room”; the room directive now asserts the real room and forbids the “nuclear reactor” misread. The world model itself was always correct — the question was reaching a context-free path. - **Terminal is real; mock leak fenced.** Confirmed `RUN_CMD` uses a real `subprocess.run` (`executor_enhanced.py:5350`) behind the destructive-command security gate — no production mock path. The <MagicMock ...> rows in the live failures DB came from `tests/test_shell_security_gate.py` patching `subprocess.run` (a test-isolation slip); a guard in `SelfImprovementEngine.log_failure` now drops any `Mock/MagicMock` repr at the write source. - **Self-repair:** self-patch no longer hallucinates a file target for non-file failures (returns `no_groundable_file`); the live self-model block no longer leaks into user-facing `SELF_ANALYZE`; stale/test-leak failure tallies purged. LoRA pipeline confirmed green end-to-end (blocked only on human dataset review, by design).

Update Advisory — 2026-06-08 (autonomy, persona voice, full-brain audit)

Full cognition audit + the gaps it closed (all on main, tested): - **Goal autogenesis** (eli/planning/goal_autogenesis NEW) — the autonomy stack (goal_store → due_goals → governed_goal_tick → proposal_queue) was fully wired but the goal store was always EMPTY (create_goal was operator-only), so ELI only reacted, never set his own agenda. Now propose_goals_from_signals turns the signals he already computes each proactive tick — world-model awareness suggestions (≥ 0.6), recurring failures ($\geq 5\times$), and code-health improvements (oversized fns / dup blocks in his own god-files) — into **proposal_only goals** (surface for approval via the existing path; never silent execution), deduped by stable id, capped at 6, logged as self_goal observations. Wired into proactive_daemon. His agenda now spans failure-repair, world-driven, AND self-improvement. - **Persona reaches factual answers** — the compact grounded-synthesis path (_compact_ground_synthesis) ran a persona-less prompt, so grounded/introspection answers were a flat data terminal. Now carries a VOICE primer pulled live from the canonical persona (stays in sync), character delivery-only; the EXACT-FACTS contract still binds every number/path/table to the evidence. Persona base itself de-bloated (6 redundant rapport blocks → 2; net smaller than before, with the new ADAPTIVE PRESENCE character + tone-adaptation). Live self-model now injected into the per-turn prompt (auto-tracks agents/ caps/model/world-room). - **Introspection is mode-gated** — EXPLAIN_MEMORY/COGNITION_RUNTIME were verbatim-in-all- modes (two different questions → same raw dump). Now: verbatim dump in quick, SYNTHESISED (persona-bound, hardened exact-facts contract) in non-quick — the blueprint's gather-then-summarise intent. - **Habits bridged** — detect_habits() filled the habits table but HABIT_STATUS/overlay read the empty habit_rules → “no habits detected” despite 1059 events. New Memory.get_detected_habits() surfaces real behaviour (meta/introspection noise filtered). - **Self-repair hardened** — self-patch no longer hallucinates a file target for non-file failures (returns no_groundable_file); FAISS index-corruption bug fixed (pickle→JSON meta); name surfaced in the personal-memory path; KG populator enriched. All recurring-failure tallies (11434 / No-commands / Missing-desc / LoRA / UnboundLocal) fixed at source + cleared. - **LoRA pipeline confirmed green** end-to-end (preflight→build_job→train→eval ok); will_train is False only because the 615-row dataset awaits human review — the correct human-gated resting state, not a failure. - **Audit verdict:** the cognition pipeline is genuinely wired, not stubbed — 14 real agents, 4 distinct reasoning modes, engagement→mode-escalation live, world-model acts at ≥ 0.70 , DAG topological layers, real self-improvement loop. No NotImplementedError/stub bodies in cognition/kernel/planning/runtime. The intelligence ceiling is the loaded model.

Verdict

The architecture is genuinely frontier for a local, model-agnostic, self-honest personal AI — it tests itself, evals itself nightly, writes its own tests, grounds its generation in real evidence, resolves intent with the model against its own catalogue, hears its wake word over music with a model it trained itself, and reads the user's vocal tone into its responses — all on the GPU, all local. The open work is engineering debt (god-files, swallowed exceptions, the verbatim/routing logic duplicated across GUI/engine/ router) and the model ceiling — both known, both phased, neither blocking.

Update — 2026-06-09 (examiner/grounding correctness; VRAM; clean shutdown)

Stats verified: ~134k LOC / 353 files / 206 capabilities / 14 bus agents / 6,659 tests collected (152 test files). This session's fixes (all on main, tested): - **Deterministic reports verbatim:** EXAMINE_CODE / FILE_AUDIT / FIX_FILE are no longer re-narrated by the quick chat path — killed three live confabulation classes (FILE_AUDIT inventing 5 nonexistent files; EXAMINE_CODE inventing findings that were existing comments; FIX_FILE narrating “I cannot fix” after it had written the file). - **Examiner correctness:** star-import false positives filtered (~800→0); tier-3 reviews the whole file in line-correct windows (was first ~300 lines) and rejects out-of-window line guesses. - **Routing:** “issues with the files” → EXAMINE_CODE (was FILE_AUDIT); “fix the bugs in X” stays FIX_FILE. FIX_FILE recovers the last-referenced file for a bare “fix it”. - **VRAM:** STT VRAM-aware (CPU on ≤ 8 GB) → main model gpu_layers 11→99; scheduled nightly jobs deduped (4× testgen/3× eval → 1 each). - **Clean shutdown:** os._exit-after-flush bypasses the llama.cpp/FAISS teardown segfault. - **GUI drag-drop** preserves the full file path so file actions resolve on disk.